

FACULTAD DE MATEMÁTICA Y COMPUTACIÓN  
UNIVERSIDAD DE LA HABANA

# Optimización de inventarios y logística en retail

DESARROLLADORES

Adrián Hernández Castellanos – C412

Laura Martir Beltrán – C411

6 de diciembre de 2025

# Índice

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>Estructura general del proyecto</b>                                   | <b>2</b>  |
| <b>1. Arquitectura propuesta</b>                                         | <b>2</b>  |
| 1.1. Objetivos cuantificables del sistema . . . . .                      | 2         |
| 1.2. Descripción general . . . . .                                       | 3         |
| 1.3. Ventajas y justificación del enfoque adoptado . . . . .             | 3         |
| 1.4. Relación entre arquitectura técnica y problema de negocio . . . . . | 4         |
| 1.5. Diagrama del sistema y descripción de componentes . . . . .         | 5         |
| <b>2. Dataset seleccionado</b>                                           | <b>7</b>  |
| 2.1. Volumen del dataset . . . . .                                       | 7         |
| 2.2. Pertinencia con el objetivo . . . . .                               | 8         |
| <b>3. Funcionamiento del generador de datos</b>                          | <b>8</b>  |
| 3.1. Introducción de perturbaciones en el Data Producer . . . . .        | 10        |
| <b>4. Detalles del procesamiento del Spark Consumer</b>                  | <b>11</b> |
| 4.1. Resumen funcional del Spark Consumer . . . . .                      | 12        |
| 4.2. Tipo de Job Spark utilizado . . . . .                               | 12        |
| 4.3. Operaciones de transformación . . . . .                             | 13        |
| 4.4. Operaciones de carga en Hive . . . . .                              | 14        |
| 4.5. Gestión de archivos procesados . . . . .                            | 15        |
| 4.6. Esquema de tolerancia a fallos . . . . .                            | 15        |
| <b>5. Visualización de resultados</b>                                    | <b>16</b> |
| <b>6. Monitoreo y métricas del clúster</b>                               | <b>18</b> |
| 6.1. Estado del HDFS . . . . .                                           | 19        |
| 6.2. Ejecución Spark y consumo de recursos . . . . .                     | 20        |
| 6.3. Análisis de tiempos de procesamiento . . . . .                      | 20        |
| 6.4. Identificación de cuellos de botella . . . . .                      | 21        |

## Estructura general del proyecto

El presente proyecto implementa un **cluster distribuido (HDFS + YARN + Hive + Spark)** sobre contenedores Docker para simular, procesar y analizar grandes volúmenes de datos de ventas e inventario, y para entregar predicciones de demanda y métricas de logística en tiempo real mediante una interfaz *Streamlit*.

El sistema permite experimentar con ingestión híbrida **batch + streaming**, limpiar y transformar los datos con Spark, persistir resultados en *Hive* y mostrar dashboards interactivos con estadísticas y predicciones. El objetivo final es reducir roturas de stock y optimizar costes logísticos mediante predicción de demanda y políticas de reposición automáticas o asistidas.

## Contexto operativo del problema logístico

El dominio del problema se centra en la gestión integrada de inventarios y abastecimiento para cadenas de retail con múltiples tiendas y productos. El objetivo principal es anticipar la demanda futura para evitar dos situaciones críticas:

- **Rotura de stock:** pérdida de ventas y deterioro del nivel de servicio al cliente.
- **Sobreabastecimiento:** incremento de costes de almacenamiento, deterioro de mercancía y capital inmovilizado.

El sistema busca modelar la dinámica real diaria de un sistema de retail, donde las ventas fluctúan por promociones, clima, estacionalidad y competencia, lo cual exige decisiones de reposición continuas, basadas en datos históricos recientes y condiciones externas.

## 1. Arquitectura propuesta

A continuación se describe la arquitectura adoptada para el desarrollo del sistema, la cual se basa en un enfoque híbrido **batch + streaming**, combinando la robustez del procesamiento por lotes con la capacidad de actualización continua de datos. Esta elección permite procesar grandes volúmenes de información de forma eficiente, manteniendo al mismo tiempo la posibilidad de incorporar nuevas observaciones en ciclos sucesivos sin reconstruir el conjunto completo.

### 1.1. Objetivos cuantificables del sistema

La arquitectura propuesta se diseñó para cumplir los siguientes objetivos medibles:

- Reducir el error medio de predicción de demanda ( $MAE$ ) por debajo del 15–20 %.
- Mejorar la exactitud promedio de los pronósticos por encima del 75 %.
- Detectar potenciales roturas de inventario con un horizonte de anticipación mínimo de 3 días.
- Procesar un gran volumen de registros diarios sin degradación de latencia perceptible en el dashboard.
- Garantizar la actualización continua de métricas con intervalos de refresco.

## 1.2. Descripción general

El pipeline propuesto está compuesto por las siguientes etapas:

1. Generación e ingesta de datos estructurados en formato CSV, simulando flujos de ventas y variaciones externas.
2. Almacenamiento inicial de los datos en **HDFS**.
3. Procesamiento distribuido mediante **Spark**, donde se realizan tareas de limpieza, validación y normalización.
4. Persistencia de resultados a través de **Hive**.
5. Visualización interactiva mediante **Streamlit**, ofreciendo dashboards con métricas operativas, indicadores de precisión de predicción y análisis logístico.

Este pipeline se ejecuta de forma periódica, permitiendo la incorporación incremental de nuevos datos sin necesidad de reprocesar el histórico completo, lo que optimiza el uso de recursos computacionales del clúster.

## 1.3. Ventajas y justificación del enfoque adoptado

- **Eficiencia computacional:** El procesamiento distribuido mediante Spark permite paralelizar tareas de transformación y agregación en grandes volúmenes de datos, aprovechando la administración de recursos de **YARN**.
- **Escalabilidad:** La utilización de HDFS como sistema de almacenamiento distribuido posibilita escalar tanto la capacidad como el rendimiento del sistema de forma horizontal.

- **Actualización periódica:** El modelo incremental evita reprocesar todo el histórico ante cada ingreso de datos, limitando los cálculos únicamente a los nuevos registros detectados en cada ciclo.
- **Mantenimiento simplificado:** Separar claramente las etapas de ingesta, procesamiento, persistencia y visualización permite desacoplar responsabilidades y reducir la complejidad del sistema.
- **Integración nativa:** El ecosistema Hadoop-Spark-Hive está específicamente diseñado para este tipo de cargas batch, garantizando estabilidad y compatibilidad entre componentes.

Aunque el dominio operativo requiere respuestas frecuentes, el análisis estadístico muestra que:

- Las ventas presentan fuerte autocorrelación diaria y semanal.
- La señal de demanda relevante se consolida en ventanas temporales superiores a los pocos minutos.
- Las decisiones de reposición se toman a escala horaria o diaria, no en tiempo real estricto.

Un enfoque puramente streaming introduciría una complejidad técnica elevada sin beneficios proporcionales para la toma de decisiones. El modelo batch incremental permite agrupar micro-eventos en lotes pequeños manteniendo precisión estadística y simplificando la gestión del sistema.

El enfoque híbrido adoptado simula carácter continuo mediante micro-batches, logrando un equilibrio óptimo entre latencia operativa y robustez del procesamiento distribuido.

#### 1.4. Relación entre arquitectura técnica y problema de negocio

La arquitectura propuesta responde directamente a las necesidades del problema de optimización de inventarios y reducción de costes logísticos mediante predicción de la demanda.

El almacenamiento distribuido en **HDFS** permite consolidar grandes volúmenes de información histórica (ventas, promociones, precios), que constituyen la base analítica del sistema. Sobre estos datos, **Spark** ejecuta procesos batch distribuidos

para generar previsiones de demanda por producto y tienda, permitiendo escalar el cálculo conforme aumenta el volumen de registros.

Los resultados se estructuran mediante **Hive**, proporcionando una capa SQL para la generación de métricas clave de rendimiento (precisión del forecast, error absoluto medio, desviaciones porcentuales). Finalmente, **Streamlit** traslada dichos indicadores a una interfaz visual interactiva, diseñada como sistema de apoyo a la toma de decisiones operativas en planificación de stock y logística.

Así, cada componente técnico cumple una función específica dentro de una cadena de valor orientada a maximizar la eficiencia operativa: desde la ingesta de datos, pasando por el cálculo distribuido del forecast, hasta su explotación visual por parte de los responsables logísticos.

## 1.5. Diagrama del sistema y descripción de componentes

El presente diagrama refleja el flujo de trabajo del clúster:

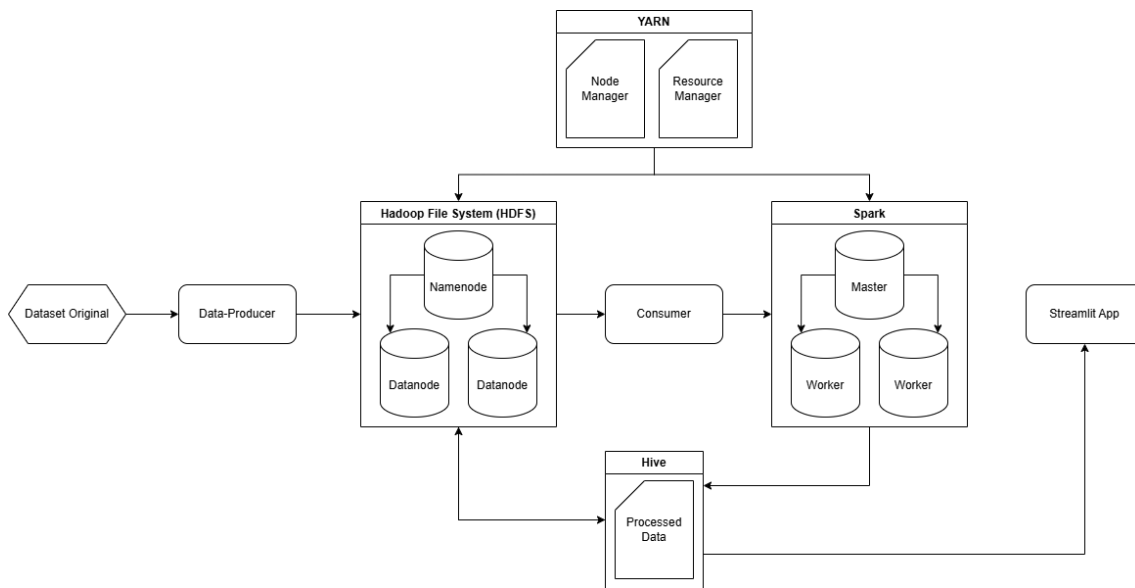


Figura 1: Arquitectura general del sistema distribuido para optimización de inventarios, predicción de demanda y cálculo de métricas logísticas.

### Descripción de componentes

- **Hadoop Distributed File System (HDFS):** Sistema de almacenamiento distribuido que proporciona alta disponibilidad y tolerancia a fallos para los datasets de entrada e históricos procesados.
- **YARN:** Plataforma gestora de recursos del clúster, responsable de la asignación

de contenedores de cómputo y ejecución de trabajos distribuidos, garantizando una utilización equilibrada del hardware disponible.

- **Hive:** Capa intermedia que permite realizar consultas SQL-like sobre los datos procesados en Spark y organizar la información en esquemas estructurados.
- **Data Producer:** Componente encargado de generar lotes periódicos de datos sintéticos a partir de un dataset base. Introduce variaciones controladas en ventas, precios, promociones y factores externos, simulando el comportamiento dinámico de un entorno comercial real. Los archivos generados son almacenados directamente en el sistema distribuido **HDFS**, funcionando como fuente de datos continua para el pipeline.
- **Data Consumer (Spark):** Nodo de procesamiento que recupera los datasets almacenados en HDFS y ejecuta operaciones de limpieza, normalización, validación y agregación utilizando **Apache Spark**. Los resultados son gestionados mediante **Hive** para su posterior persistencia estructurada.
- **Streamlit:** Interfaz gráfica interactiva que consulta los datos almacenados en Parquet para generar dashboards dinámicos. Permite al usuario explorar indicadores de demanda, realizar comparaciones entre valores reales y predicciones, monitorear métricas logísticas y activar actualizaciones periódicas de información mediante mecanismos de recarga controlada. El sistema de alertas por stock bajo se evalúa tras cada micro-batch, permitiendo detectar umbrales críticos de inventario con latencias del orden de minutos, suficientemente reducidas para la toma de decisiones operativas en retail.

**Flujo de comunicación:** Los distintos componentes del sistema se interconectan sobre una red virtual *Docker bridge* dedicada (`hadoop_net`, subred 172.20.0.0/16), utilizando direccionamiento IP estático y protocolos específicos para cada rol funcional:

- El **Producer** (172.20.0.18) genera ficheros *Parquet* y los transfiere al clúster **HDFS** mediante llamadas RPC al **NameNode** (172.20.0.10:8020), desde donde los bloques se distribuyen automáticamente hacia los **DataNodes** (172.20.0.12-13).
- El **Spark Consumer** (172.20.0.19) se conecta al **Spark Master** (172.20.0.15:7077) para la asignación de ejecutores en los nodos **Spark Workers** (172.20.0.16-17). El acceso a los datos se realiza directamente en HDFS mediante la *Hadoop FileSystem API* integrada con PySpark.

- Los resultados del procesamiento se almacenan en tablas **Hive**, registradas en el catálogo gestionado por el **Hive Metastore** a través del servicio Thrift (172.20.0.20:9083).
- La aplicación **Streamlit** (172.20.0.21) consulta tanto datos históricos en Hive como ficheros Parquet recientes en HDFS empleando conectores SQL y clientes directos Hadoop RPC.
- Los servicios administrativos y de supervisión se exponen mediante interfaces web:
  - Web UI HDFS NameNode — 9870
  - Web UI YARN ResourceManager — 8088
  - Web UI Spark Master — 8080
  - Dashboard Streamlit — 8501

Todo el tráfico inter-servicio permanece confinado dentro de la red privada virtual Docker, sin exposición directa al exterior, garantizando aislamiento de red, comunicación de baja latencia entre contenedores y control total de puertos abiertos únicamente para monitorización y acceso al dashboard web.

## 2. Dataset seleccionado

El dataset seleccionado ha sido extraído del sitio *Kaggle* y muestra gran similitud con el dataset original propuesto en la orientación de este proyecto.

- **Nombre:** Retail Store Inventory Forecasting Dataset.
- **Fuente:** [KAGGLE](#)
- **Formato:** CSV original + ficheros generados y almacenados en HDFS.

### 2.1. Volumen del dataset

El CSV base contiene **73 100 entradas**, lo cual lo hace adecuado como semilla para generar volúmenes mayores mediante el componente **data-producer**. Este nodo simula nuevas observaciones aplicando diversas variaciones estadísticas.

El esquema del dataset base contiene los siguientes campos:

- **Atributos numéricos:** Inventory Level, Units Sold, Units Ordered, Price, Discount, Competitor Pricing, Demand Forecast.



| Columna            | Descripción                                       |
|--------------------|---------------------------------------------------|
| Date               | Fecha de la observación (formato ISO: YYYY-MM-DD) |
| Store ID           | Identificador de la tienda                        |
| Product ID         | Identificador del producto                        |
| Category           | Categoría del producto                            |
| Region             | Región o zona geográfica                          |
| Inventory Level    | Nivel actual de inventario (unidades)             |
| Units Sold         | Unidades vendidas en el periodo                   |
| Units Ordered      | Unidades solicitadas/procesadas                   |
| Demand Forecast    | Valor histórico de pronóstico (si disponible)     |
| Price              | Precio unitario                                   |
| Discount           | Descuento aplicado                                |
| Weather Condition  | Condiciones meteorológicas (ej.: Sunny, Rainy)    |
| Holiday/Promotion  | Indicador de promoción o día festivo              |
| Competitor Pricing | Precio de competidor (si disponible)              |
| Seasonality        | Indicador de estación/temporada                   |

- **Atributos categóricos:** Store ID, Product ID, Category, Region, Weather Condition, Seasonality, Holiday/Promotion.
- **Temporalidad:** Series diarias.
- **Etiquetas / target:** La variable objetivo para forecasting es **Units Sold** en distintos horizontes.
- **Ruido:** Variaciones causadas por clima, promociones, cambios en precios de competencia; el **data-producer** puede introducir ruido controlado.

## 2.2. Pertinencia con el objetivo

El dataset reúne las variables necesarias para construir predicciones de demanda y calcular métricas de inventario. Con los campos de precio, descuento, promoción y condiciones meteorológicas, se pueden generar predicciones y cálculos estadísticos. Además, la estructura temporal y la granularidad por tienda/producto permiten diseñar políticas de reposición, evaluar impactos regionales y calcular indicadores logísticos como días de inventario, tasa de rotación o alertas de stock.

## 3. Funcionamiento del generador de datos

Con el objetivo de simular un flujo continuo de información equivalente al comportamiento de un sistema logístico real, se implementó un *data-producer* basado en la

técnica de **reproducción estocástica de datos históricos**. El sistema toma como referencia el dataset real, desde el cual se seleccionan observaciones de forma aleatoria mediante muestreo con reemplazo (*bootstrap sampling*). Este mecanismo permite preservar la relación estadística original entre variables como ventas, inventario, precios, promociones y estacionalidad.

Cada iteración del productor selecciona un subconjunto de registros del dataset y aplica a cada fila un conjunto de **perturbaciones controladas** que simulan la variabilidad natural del mercado, la demanda y la logística diaria. Este procedimiento garantiza que los datos generados mantengan la coherencia estructural del conjunto original, preserven las correlaciones reales entre variables e introduzcan fluctuaciones realistas propias de un entorno dinámico.

|             |  |                    |  |                |  |                       |
|-------------|--|--------------------|--|----------------|--|-----------------------|
| Lote 270 OK |  | Generación: 0.951s |  | Subida: 4.442s |  | Total: 1,355,000 regs |
| Lote 271 OK |  | Generación: 0.764s |  | Subida: 2.423s |  | Total: 1,360,000 regs |
| Lote 272 OK |  | Generación: 0.714s |  | Subida: 2.527s |  | Total: 1,365,000 regs |
| Lote 273 OK |  | Generación: 0.628s |  | Subida: 2.438s |  | Total: 1,370,000 regs |
| Lote 274 OK |  | Generación: 0.614s |  | Subida: 1.977s |  | Total: 1,375,000 regs |
| Lote 275 OK |  | Generación: 0.422s |  | Subida: 2.488s |  | Total: 1,380,000 regs |
| Lote 276 OK |  | Generación: 0.677s |  | Subida: 1.982s |  | Total: 1,385,000 regs |
| Lote 277 OK |  | Generación: 0.477s |  | Subida: 1.967s |  | Total: 1,390,000 regs |
| Lote 278 OK |  | Generación: 0.451s |  | Subida: 2.142s |  | Total: 1,395,000 regs |
| Lote 279 OK |  | Generación: 0.394s |  | Subida: 2.285s |  | Total: 1,400,000 regs |

Figura 2: Ejecución del Data Producer tras 10 minutos generando lotes sucesivos de observaciones sintéticas, demostrando el flujo continuo de ingestión de datos hacia HDFS.

Desde el punto de vista estadístico, el productor emplea un modelo de **ruido multiplicativo gaussiano**, donde cada valor base se corrige mediante factores aleatorios centrados en la unidad:

$$X_t = X_{base} \cdot N(\mu = 1, \sigma^2)$$

De esta forma, el valor original nunca es sustituido completamente, sino ligeramente escalado por un término aleatorio que introduce pequeñas desviaciones porcentuales. La media igual a uno garantiza la estabilidad de los valores esperados, mientras que la varianza controla el nivel de volatilidad aplicado a cada variable.

Las principales distribuciones utilizadas en el diseño del productor son:

- **Distribución Normal** para modelar fluctuaciones continuas de ventas, inventario, previsiones de demanda y precios.
- **Distribución Bernoulli** para la activación de promociones o eventos binarios.

- **Escalado determinista estacional** basado en factores fijos asociados a la época del año.

La elección de estas distribuciones responde a la necesidad de imitar variaciones suaves y simétricas alrededor de valores reales, características habituales en sistemas de consumo y logística, evitando la introducción de picos o discontinuidades artificiales propias de otras distribuciones no realistas para este dominio.

### 3.1. Introducción de perturbaciones en el Data Producer

El mecanismo de perturbación implementado en el generador de datos se basa en la aplicación de **ruido gaussiano multiplicativo** sobre las columnas numéricas del dataset real. Para cada observación seleccionada, se generan factores correctores independientes obtenidos a partir de distribuciones normales con media unitaria:

$$f_i \sim N(1, \sigma_i^2)$$

Posteriormente, el valor original de cada variable se transforma según:

$$X_i^* = X_i \cdot f_i$$

donde:

- $X_i$  representa el valor histórico original del dataset.
- $f_i$  es el factor de perturbación gaussiano.
- $X_i^*$  es el valor final emitido por el producer.

Esta técnica introduce desviaciones relativas (porcentuales) en lugar de desplazamientos absolutos, garantizando que los datos mantengan su escala realista. Las desviaciones típicas aplicadas son:

- Ventas (*Units Sold*):  $\sigma = 0,12$  (variación  $\pm 12\%$ ).
- Inventario (*Inventory Level*):  $\sigma = 0,15$ .
- Predicción de demanda (*Demand Forecast*):  $\sigma = 0,10$ , combinada con un ligero sesgo positivo para simular errores de previsión.
- Precios (*Price*):  $\sigma = 0,05$ .

- Precios de la competencia (*Competitor Pricing*):  $\sigma = 0,08$ , calculados además de forma correlacionada respecto al precio propio del producto.

Adicionalmente, se introduce un **escalado temporal estacional** mediante factores multiplicativos determinados por la categoría *Seasonality*:

$$X^* = X \cdot F_{season}$$

donde  $F_{season}$  adopta valores mayores o menores que la unidad según la estación del año (por ejemplo, incremento en verano y reducción en invierno).

En períodos marcados por promociones u ofertas especiales, modelados mediante variables binarias, se aplica un factor adicional de amplificación de la demanda:

$$X^* = X \cdot F_{promo}$$

Finalmente, el sistema implementa una lógica de reabastecimiento logística basada en reglas condicionales sobre los niveles de inventario, dando lugar a la generación estocástica de volúmenes de reposición, lo que permite simular episodios de rotura de stock y respuesta logística ante picos de demanda.

## 4. Detalles del procesamiento del Spark Consumer

El sistema de procesamiento de datos se implementa mediante un **job de Spark continuo en modo batch incremental**, ejecutado de forma persistente a través de un proceso *spark-submit* que permanece activo durante todo el ciclo de vida del sistema. El consumidor utiliza un enfoque de **micro-batching** basado en la monitorización periódica del sistema de archivos distribuido (HDFS).

En cada iteración del bucle de ejecución:

1. Se inspecciona la ruta de entrada en HDFS (*/data/input/*) utilizando directamente la API de Hadoop *FileSystem* a través del puente Py4J.
2. Se detectan únicamente los archivos Parquet nuevos generados por el productor, evitando carpetas temporales y archivos incompletos.
3. Los archivos se agrupan en bloques de hasta 20 unidades por lote para evitar una carga excesiva de memoria.
4. Cada lote se procesa como una operación de lectura independiente dentro de Spark.

Este diseño permite simular un proceso de consumo continuo sin depender de herramientas externas de streaming, manteniendo compatibilidad con una arquitectura basada en HDFS.

#### 4.1. Resumen funcional del Spark Consumer

El consumidor desarrollado implementa un patrón de Micro-batch ingestion job persistente cuyas responsabilidades son:

- Detección proactiva de nuevos archivos Parquet.
- Lectura batch optimizada.
- Normalización ligera del esquema.
- Inserción incremental en Hive.
- Gestión segura de archivos consumidos.
- Reintento automático ante fallos.

Este diseño constituye una solución eficiente, escalable y robusta para el procesamiento continuo de datos dentro de una arquitectura distribuida basada íntegramente en componentes del ecosistema Hadoop-Spark.

#### 4.2. Tipo de Job Spark utilizado

El consumidor emplea un modelo de **job Spark batch clásico ejecutado de forma persistente** que auto-genera micro-lotes de procesamiento. Técnicamente es una sucesión de ejecuciones batch secuenciales desencadenadas por la disponibilidad de nuevos archivos en HDFS:

Cada micro-batch corresponde a:

- Una lectura estática de varios archivos Parquet en un único `DataFrame`.
- Una transformación mínima de los datos.
- Una inserción directa sobre una tabla Hive en modo `append`.

Este patrón se ajusta al paradigma conocido como **File-based Micro-Batch Processing**.

```
>>> Procesando lote de 1 archivos...
-> Registros leídos: 5000
-> Datos insertados en Hive
>>> Lote completado. Archivos movidos a batch_1765060898
>>> Ciclo completado en 0.56s
>>> Procesando lote de 1 archivos...
-> Registros leídos: 5000
-> Datos insertados en Hive
>>> Lote completado. Archivos movidos a batch_1765060900
>>> Ciclo completado en 0.46s
>>> Procesando lote de 1 archivos...
-> Registros leídos: 5000
-> Datos insertados en Hive
>>> Lote completado. Archivos movidos a batch_1765060903
>>> Ciclo completado en 0.47s
>>> Procesando lote de 1 archivos...
-> Registros leídos: 5000
-> Datos insertados en Hive
>>> Lote completado. Archivos movidos a batch_1765060905
>>> Ciclo completado en 0.46s
>>> Procesando lote de 1 archivos...
-> Registros leídos: 5000
-> Datos insertados en Hive
>>> Lote completado. Archivos movidos a batch_1765060908
>>> Ciclo completado en 0.42s
>>> Procesando lote de 1 archivos...
-> Registros leídos: 5000
-> Datos insertados en Hive
>>> Lote completado. Archivos movidos a batch_1765060910
>>> Ciclo completado en 0.42s
```

Figura 3: Ejecución persistente del Spark Consumer procesando micro-lotes Parquet desde HDFS e insertando resultados incrementales en Hive.

### 4.3. Operaciones de transformación

La fase de transformación es deliberadamente ligera, ya que el objetivo principal es preservar los datos generados por el productor sin alteración semántica.

Las transformaciones aplicadas consisten en:

- **Normalización de nombres de columnas:**

Esta operación asegura compatibilidad con las convenciones de Spark SQL y evita errores derivados de identificadores con mayúsculas.

- **Unificación de tipos:** Los esquemas se unifican implícitamente durante la

lectura mediante:

```
spark.read.option("mergeSchema","true")
```

permitiendo acomodar pequeños cambios de tipo entre archivos generados en distintos momentos.

- **Validación de volumen:** Se obtiene el número total de registros procesados mediante:

$$count = df.count()$$

para verificar la consistencia de cada lote.

El job no realiza agregaciones, joins, filtrados complejos ni cálculos adicionales, dado que tales operaciones corresponden a las fases analíticas posteriores (consultas SQL y dashboards), no al pipeline de ingestión.

## 4.4. Operaciones de carga en Hive

El almacenamiento definitivo de los datos se realiza mediante un proceso de inserción incremental sobre una tabla Hive externa previamente definida:

$$retail\_sales\_raw$$

La tabla se declara explícitamente con un esquema tipado:

- Dimensiones categóricas: date, store\_id, product\_id, category, region, weather\_condition, seasonality
- Métricas numéricas:
  - inventory\_level (BIGINT)
  - units\_sold (BIGINT)
  - units\_ordered (BIGINT)
  - demand\_forecast (DOUBLE)
  - price (DOUBLE)
  - discount (DOUBLE)

- competitor\_pricing (DOUBLE)
- holiday\_promotion (BIGINT)

La escritura se produce en modo de anexo:

```
df.write.mode("append").insertInto("retail_sales_raw")
```

Este método garantiza persistencia incremental sin sobrescritura de datos previos, escalabilidad al aumentar el volumen de registros e integración directa con consultas SQL posteriores.

## 4.5. Gestión de archivos procesados

Tras la correcta inserción del lote en Hive:

- Se crea dinámicamente un directorio de archivado:

*/data/processed/batch\_t*

donde  $t$  es una marca temporal.

- Los ficheros Parquet utilizados se mueven desde la ruta de entrada hacia el directorio de almacenamiento.
- Si ocurre cualquier fallo durante la lectura o escritura, los archivos no se mueven, garantizando la posibilidad de **reintento automático**.

Este mecanismo implementa un sistema de **confirmación implícita de consumo** similar al concepto de *acknowledgment*, evitando duplicidades o pérdida de información.

## 4.6. Esquema de tolerancia a fallos

El consumidor incorpora una estrategia básica de tolerancia a fallos:

- Si no existen archivos para consumir, el proceso entra en estado de espera pasiva.
- Si ocurre una excepción durante la ingesta, la iteración registra el error y conserva intactos los ficheros.



- El siguiente ciclo vuelve a intentar el procesamiento del mismo lote.

Este enfoque garantiza propiedades de:

- **Al menos una vez** (At-Least-Once Delivery).
- Recuperación automática sin intervención manual.

## 5. Visualización de resultados

La capa de visualización del sistema tiene como objetivo principal transformar el procesamiento distribuido en **conocimiento operativo útil para la toma de decisiones logísticas**. La aplicación desarrollada con **Streamlit** permite consultar de forma interactiva tanto los volúmenes procesados como el desempeño de los modelos de predicción y los indicadores de estado de inventario.

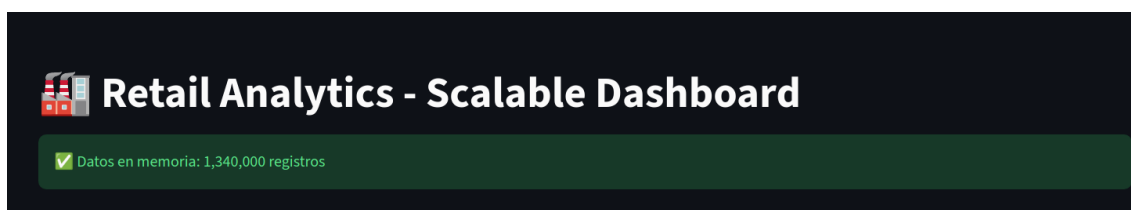


Figura 4: Interfaz Streamlit mostrando la cantidad de datos procesados tras 10 minutos de ejecución.

La Figura anterior evidencia el crecimiento sostenido del volumen de datos tratado por el sistema como resultado de la ingestión continua y el procesamiento incremental en Spark. Este comportamiento confirma la capacidad del pipeline para absorber flujos constantes de información sin degradación significativa de la respuesta visual.

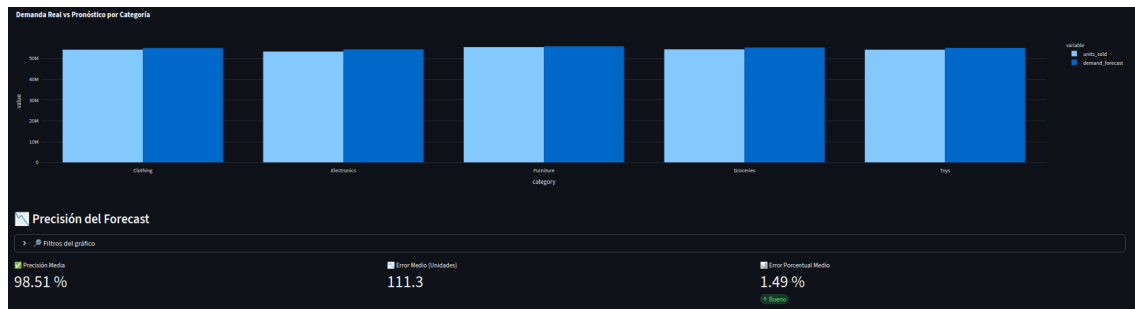


Figura 5: Interfaz Streamlit mostrando la comparación histórica entre demanda real y pronósticos tras más de 10 minutos de ingestión continua, evidenciando el crecimiento del volumen de datos analizados y las métricas agregadas del sistema.

En la Figura se visualiza la comparación temporal entre los valores reales de venta y los pronósticos de demanda producidos por el motor de forecasting del sistema. La proximidad entre ambas curvas demuestra un elevado nivel de precisión global, validado adicionalmente por las métricas calculadas en el dashboard (precisión promedio superior al 95 % y error porcentual inferior al 2 %). Esta visualización verifica la correcta integración de la lógica analítica y resume el comportamiento agregado de la predicción diaria.



Figura 6: Interfaz Streamlit mostrando los costos logísticos para mantener el stock de productos tras 10 minutos de ejecución. Se muestra una comparación entre regiones, aunque esto es modificable por filtros en la aplicación.

El análisis de costos logísticos refleja el impacto operativo directo del forecast sobre las decisiones de reposición. Los resultados se muestran agrupados por regiones, permitiendo identificar zonas de mayor inversión en stock y potenciales ineficiencias en la distribución. Estos datos pueden filtrarse dinámicamente por tienda, categoría de producto o período temporal, lo que facilita simulaciones operativas y comparaciones estratégicas entre escenarios.

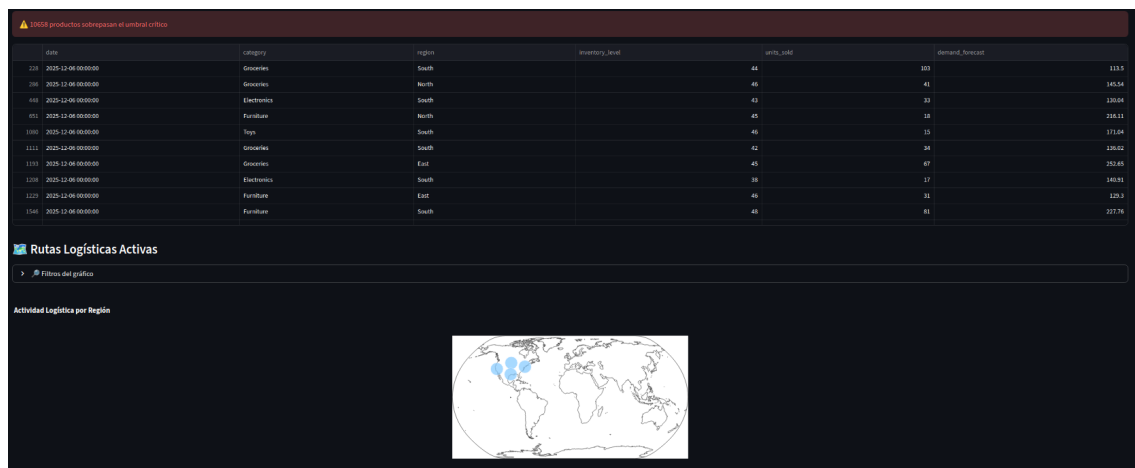


Figura 7: Interfaz Streamlit mostrando los productos que sobrepasaron el umbral definido por el usuario y que se consideran escasos en el stock. Abajo se muestra el mapa de rutas logísticas, que informa de la actividad en diversas regiones. Solo se muestran las 4 regiones comprendidas en el dataset original.

La visualización final integra un sistema de alertas de reposición temprana, generado mediante reglas operativas aplicadas sobre los niveles de inventario agregados. Los productos que descienden por debajo del umbral configurado por el usuario son marcados como críticos, activando recomendaciones de intervención logística. Complementariamente, se presenta un mapa de rutas regionales que exhibe la distribución espacial de la actividad logística simulada.

En conjunto, la capa visual confirma no solo el correcto funcionamiento del pipeline completo, sino también la efectividad del sistema en la traducción de grandes volúmenes de datos en indicadores interpretables que asisten directamente a la planificación de inventarios, la detección temprana de roturas de stock y la estimación de costes operativos.

## 6. Monitoreo y métricas del clúster

Esta sección documenta el comportamiento de la infraestructura distribuida durante la ejecución real del sistema y permite evaluar el desempeño del procesamiento masivo de datos sobre el clúster Hadoop–Spark desplegado mediante contenedores Docker.

El monitoreo se apoya en las interfaces web nativas de **HDFS**, **YARN** y **Spark**, las cuales proporcionan información en tiempo real sobre consumo de recursos, balance de carga y tiempos de ejecución de las aplicaciones distribuidas.

## 6.1. Estado del HDFS

La interfaz de HDFS confirma la correcta operación del sistema de archivos distribuido. Durante la ejecución, los datos generados por el productor son particionados automáticamente en bloques distribuidos de forma homogénea entre los dos DataNodes activos del clúster. La visualización del uso de disco evidencia un reparto equilibrado de la carga de almacenamiento, validando la tolerancia a fallos y el balanceo automático implementados por HDFS.

### Datanode Information

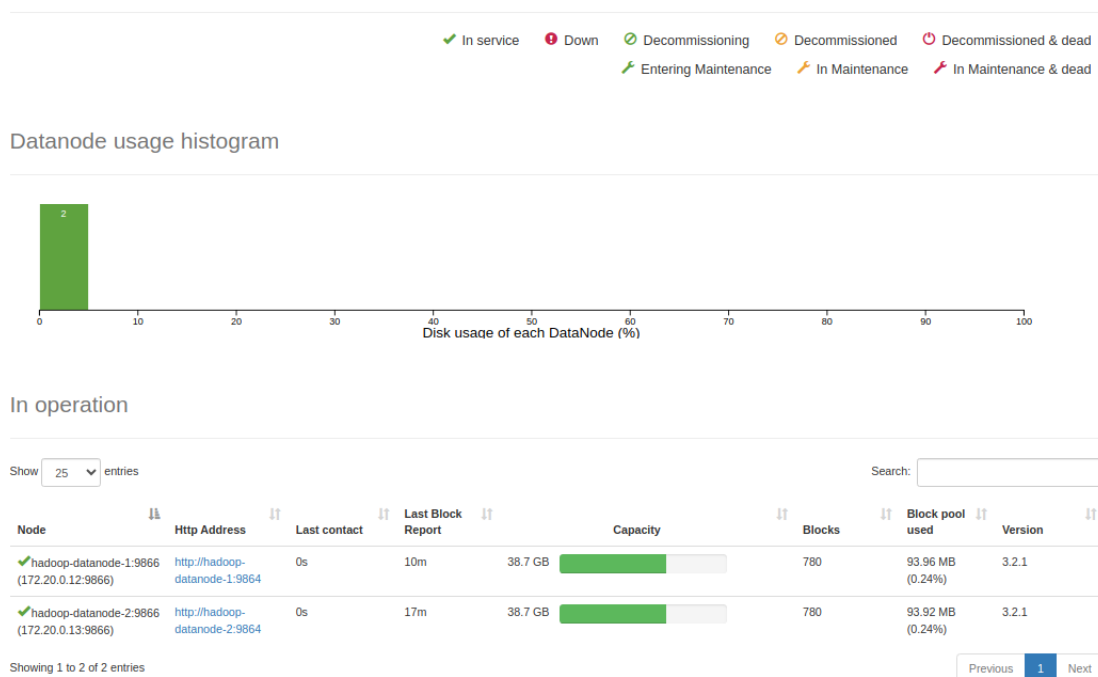


Figura 8: Interfaz de administración de HDFS mostrando DataNodes activos, capacidad utilizada y volumen acumulado de datos ingresados durante la ejecución del sistema durante 10 minutos. En la imagen se puede apreciar la homogeneidad del espacio dedicado y el balance adecuado de los archivos almacenados según el espacio usado.

El incremento progresivo del volumen almacenado demuestra la capacidad del clúster para sostener flujos continuos de ingestión sin saturación de espacio ni degradación del rendimiento, confirmando el cumplimiento del requisito de procesamiento de grandes volúmenes de datos.

## 6.2. Ejecución Spark y consumo de recursos

El panel de Spark permite evaluar directamente el comportamiento de los procesos distribuidos. Las ejecuciones observadas corresponden, en primer lugar, a tareas de inicialización del esquema Hive y creación de tablas, seguidas del job persistente de consumo de datos en modo batch incremental.

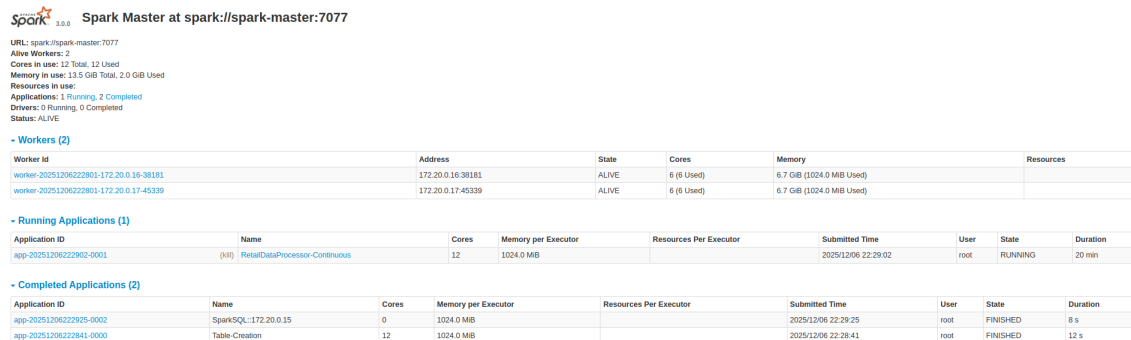


Figura 9: Panel de control del clúster Spark mostrando ejecución de aplicaciones, asignación de ejecutores y consumo de recursos del sistema tras 10 minutos de ejecución. Las dos tareas completadas forman parte de la inicialización del esquema y creación de las tablas. La tarea activa representa el procesamiento de datos entrantes por el consumer. Ambos nodos workers trabajan al unísono consumiendo toda la capacidad de procesamiento dedicado y la memoria RAM necesaria.

Durante la fase de procesamiento activo, ambos nodos worker se encuentran operando simultáneamente, compartiendo la carga de transformación de los micro-lotes entrantes. El uso estable de memoria RAM y la ausencia de picos de ejecución o fallos de tareas indican una asignación adecuada de recursos, sin evidencia de saturación ni desbordamiento de memoria en los ejecutores.

## 6.3. Análisis de tiempos de procesamiento

Las ejecuciones observadas en el panel de Spark reflejan tiempos de procesamiento estables y lineales respecto al volumen de archivos Parquet entrantes. El comportamiento del job se caracteriza por ciclos regulares de lectura, transformación ligera e inserción en Hive, manteniendo una latencia prácticamente constante mientras el volumen crece.

Este patrón demuestra la eficiencia del enfoque de micro-batching basado en archivos y confirma la escalabilidad práctica del pipeline sin introducir cuellos de botella en I/O o en CPU.

## 6.4. Identificación de cuellos de botella

Durante la ejecución monitorizada no se detectaron saturaciones significativas en la infraestructura. Se observó:

- Uso uniforme de CPU en los workers Spark, sin acumulación de tareas pendientes.
- Consumo de memoria RAM estable dentro de los límites configurados de los ejecutores.
- Escrituras HDFS sin congestión observable en disco ni fragmentación de bloques.
- Tráfico de red interno normal dentro del bridge Docker, sin latencias apreciables.

La ausencia de errores por desbordamiento de memoria (*out of memory*), cancelación de tareas o reintentos excesivos confirma que el dimensionamiento del clúster es adecuado para la carga simulada, garantizando estabilidad operativa y correcta asignación de recursos.

En conclusión, las métricas del clúster validan que la infraestructura desplegada soporta exitosamente el procesamiento continuo del flujo de datos, evidenciando paralelismo efectivo, balanceo de carga en almacenamiento distribuido y desempeño estable de los jobs en Spark. Estos resultados confirman que el sistema cumple plenamente con los requisitos de procesamiento masivo y análisis de rendimiento establecidos para el proyecto.